One place reads the environment

Animated explainer · src/config.js

The rest of the program should never wonder where a key, a model or a limit came from. config.js resolves every setting once, records its source, and hands the loop a plain object.

9 scenes · 5 config sources ·.env wins · key never logged

CONTENTS

What is in this explainer

01 · Why this file exists 3

02 · Precedence 4

03 · Finding the file 5

04 · Parsing.env 6

05 · Secrets 7

06 · Model names 8

07 · Provenance 9

08 · The object 10

09 · Recap 11

Page numbers refer to the folio at the foot of each page.

01 · Why this file exists

One reader. Every setting.

SRC/CONFIG.JS · *source*

```
const config = loadConfig({ model: opts.model });
const workspace = new Workspace(config.workspaceRoot);
const sandbox = new Sandbox({ workspace, log: console.log });
const memory = new Memory({ root: config.projectRoot, file: config.memoryFile });
```

Credentials — Key and model, with their source.

Paths — Workspace and memory file.

Bounds — Steps, timeout, policy, memory caps.

> **remember** Every environment read happens here. The rest of the program receives values and provenance.
> **On screen:** config.js is the only file that reads process.env and the only file that parses .env. It resolves credentials, paths, policies and budgets into one plain object. The loop receives the result. It never asks where a value came from.

02 · Precedence

What wins, in order.

1. **Overrides** — CLI flags
2. **.env** — nearest file
3. **Shell** — inherited
4. **Defaults** — built in

> .env deliberately wins over the shell. A stale exported key must not silently outrank the project file.
> **remember** The banner can say not just what won, but which source won.
> **On screen:** Precedence is explicit overrides, then the nearest .env, then the shell, then defaults. The .env choice is deliberate: a stale shell variable should not shadow the project file. The returned object includes source fields, so the banner can explain the decision.

03 · Finding the file

Walk up until a.env appears.

SRC/CONFIG.JS · *source*

```
export function findEnvFile(startDir = PROJECT_ROOT) {
  let dir = path.resolve(startDir);
  for (; ;) {
    const candidate = path.join(dir, '.env');
    if (fs.existsSync(candidate) && fs.statSync(candidate).isFile()) return candidate;
    const parent = path.dirname(dir);
    if (parent === dir) return null;
    dir = parent;
  }
}
```

- 📍 Start here the package root.
- ⬆️ Walk up so a repo-level .env is found.
- 🛑 Stop at root return null when the filesystem ends.

> **On screen:** A meetup repo may keep its .env beside the package or at the repository root. findEnvFile starts at the package, checks each parent, and stops at the filesystem root. The nearest .env wins. That single rule removes a lot of afternoon debugging.

04 · Parsing.env

Small parser. Clear rules.

SRC/CONFIG.JS · *source*

```
const m = line.match(/^\s*(?:export\s+)?([A-Za-z_][A-Za-z0-9_]*)\s*=\s*(.*)$/);
if (!m) continue;

const name = m[1];
const value = m[2].trim().replace(/^["'](.*)["']$/, '$1').trim();
if (!value) continue;

process.env[name] = value; // .env overrides the shell
fromFile[name] = value;
```

Comments — Lines starting with # are skipped.

Quotes — Matched surrounding quotes are stripped.

Empty — Empty values are ignored, not stored.

> The parser returns only the keys that came from the file, so provenance stays honest.
> **On screen:** The .env parser is intentionally tiny: KEY=VALUE, optional export, comments and quotes. It writes each value into process.env, and records that the value came from the file. Empty values are skipped. The returned map is what makes the source reporting possible.

05 · Secrets

Never log the key.

SRC/CONFIG.JS · *source*

```
export function fingerprint(secret) {
  if (!secret) return 'not set';
  if (secret.length <= 12) return '*'.repeat(secret.length) + ' (' + secret.length + ' chars)';
  return secret.slice(0, 5) + '...' + secret.slice(-4) + ' (' + secret.length + ' chars)';
}
```

THE BANNER · *text*

```
key      : sk-or...a1b2 (73 chars) from .env
key      : not set
```

> A prefix, a suffix and a length are enough to tell keys apart without putting the secret in a transcript.
> **On screen:** The banner needs to show which key is in use, but a key must never be logged. fingerprint keeps five characters, the last four, and the length. That is enough to debug a bad key without leaking it into a screenshot.

06 · Model names

A bare model name is not a slug.

SRC/CONFIG.JS · *source*

```
const VENDOR_BY_PREFIX = [
  [/^deepseek/i, 'deepseek'],
  [/^claude/i, 'anthropic'],
  [/^(gpt|o[13458]|chatgpt)/i, 'openai'],
  [/^gemini/i, 'google'],
  [/^grok/i, 'x-ai'],
  [/^llama/i, 'meta-llama'],
  [/^qwen/i, 'qwen'],
  [/^mistral|^mixtral/i, 'mistralai'],
];
```

> OpenRouter requires vendor/model. A .env carried from another project often holds a bare name, so the obvious prefixes are expanded instead of failing with a 400.

EXAMPLE · *text*

```
claude-sonnet-4.5  ->  anthropic/claude-sonnet-4.5
gpt-4o-mini        ->  openai/gpt-4o-mini
already/a-slug     ->  unchanged
```

> **On screen:** OpenRouter wants vendor/model, but old .env files often carry a bare name. The prefix table maps the obvious families. An explicit provider prefix overrides it. A name that already contains a slash is left alone, and the config records that it normalised.

07 · Provenance

The config remembers where it came from.

SRC/CONFIG.JS · *source*

```
return {
  apiKey,
  apiKeySource: overrides.apiKey ? 'override' : sourceOf('OPENROUTER_API_KEY', keyFromFile),
  apiKeyOverrodeShell: replaced(keyFromFile, inherited.apiKey),
  model,
  modelSource: overrides.model ? 'flag' : sourceOf('OPENROUTER_MODEL', modelFromFile),
  modelOverrodeShell: replaced(modelFromFile, inherited.model),
  modelNormalised: normalised,
  // ...paths, policy, budgets, memory...
};
```

- 🏷️ Source .env, environment, flag, override, or not set.
- ⚠️ Shadowing whether .env replaced a different shell value.
- 🔤 Normalised whether a bare model name was expanded.

> **On screen:** The config does not just return values. It returns where each important value came from. The banner can say "from .env" or "overrode the shell value" instead of guessing. That provenance is the difference between a config file and a debugging mystery.

08 · The object

One object, every knob.

apiKey / model — the live request identity.

workspaceRoot — resolved against the package root.

commandPolicy — allow, ask, or deny.

maxSteps — default 12.

requestTimeoutMs — default 90,000.

memoryFile — package-relative or absolute.

memoryMaxEntries — default 60.

memoryPromptChars — default 1,200; 0 disables injection.

> Everything downstream is pure: given this object, the rest of the program does not need to know about the environment.
> **On screen:** The returned object is the complete runtime configuration. Paths resolve now, policies are normalised, and memory caps are numbers. Downstream code stays pure because every environment decision already happened here.

09 · Recap

Resolve once. Report everything.

• 5 sources

• 1 config object

• 0 keys logged

→

Load once. No module reads process.env directly.

→

.env wins. The nearest file outranks a stale shell value.

→

Provenance matters. Return the source, not only the value.

→

Never log secrets. Fingerprint the key; never print it.

> **remember** Configuration is not a pile of environment variables. It is one resolved, explainable object.
> **On screen:** Five kinds of source, one config object, and no secret in the logs. Load once, let .env win, keep the provenance, and fingerprint the key. That is how a small program stays reproducible when the environment is not.